Bharat Electronics Limited (BEL) — SIH26127

City-Wide Multi-Camera ANPR Trajectory Tracking & Urban Traffic Intelligence Platform

Master Technical Documentation & Presentation Guide

Master Technical Documentation & Presentation Guide

City-Wide Multi-Camera Vehicle Intelligence Platform (SIH26127 — BEL)

Table of Contents

1. [Executive Summary & High-Level System Architecture](#1-executive-summary--high-level-system-architecture)

2. [Tech Stack & Frameworks Deep Dive](#2-tech-stack--frameworks-deep-dive)

3. [End-to-End Pipeline & Data Flow Architecture](#3-end-to-end-pipeline--data-flow-architecture)

4. [File-by-File Codebase & Directory Breakdown](#4-file-by-file-codebase--directory-breakdown)

5. [Docker Containerization & Deployment Setup](#5-docker-containerization--deployment-setup)

6. [SIH Hackathon Judges Q&A (Technical Defense Guide)](#6-sih-hackathon-judges-qa-technical-defense-guide)

7. [Future Upscaling & Advanced Roadmap](#7-future-upscaling--advanced-roadmap)

- [7.1 Agentic AI & Autonomous Multi-Agent Orchestration](#71-agentic-ai--autonomous-multi-agent-orchestration)
- [7.2 Ultra-Extreme Weather & Motion Blur Super-Resolution](#72-ultra-extreme-weather--motion-blur-super-resolution)
- [7.3 Zero-Trust Security, Encryption & Privacy Safeguards](#73-zero-trust-security-encryption--privacy-safeguards)

- [7.4 OSINT (Open Source Intelligence) Global Integration](#74-osint-open-source-intelligence-global-integration)

---

1. Executive Summary & High-Level System Architecture

The **City-Wide Multi-Camera ANPR Trajectory Tracking & Urban Traffic Intelligence Platform** is built to solve Bharat Electronics Limited (BEL) Problem Statement **SIH26127**.

The platform monitors multi-camera city junction video feeds to perform:

- **Real-Time Dehazing & Image Enhancement** for adverse atmospheric conditions (fog, haze, glare).
- **Vehicle Detection & Multi-Object Tracking (MOT)** across consecutive frames.
- **License Plate Detection & Optical Character Recognition (OCR)** using fine-tuned deep learning models.
- **Deep Re-Identification (Re-ID)** to correlate visual features (color, make, shape) across camera junctions without relying solely on license plates.
- **Cross-Camera Trajectory Reconstruction** to build spatial timelines of vehicle journeys.
- **Automated Security Watchlist & Anti-Spoofing Alarms** (e.g., Cloned/Duplicate Plate Detection).
- **City-Wide Urban Traffic Analytics & GIS Trajectory Line Rendering**.



---

2. Tech Stack & Frameworks Deep Dive

A. Frontend Layer (User Interface & Dashboard)

- **Next.js 16 (App Router + Turbopack)**: Modern React framework providing server-side rendering, routing, fast page builds, and optimized asset bundling.
- **TypeScript**: Ensures strong type safety across API client responses, WebSocket event payloads, and map component properties.
- **Tailwind CSS & Glassmorphism UI**: High-impact modern aesthetics featuring dark mode, dynamic indicators, and crisp visual hierarchy for security operators.
- **Leaflet.js & React-Leaflet**: Open-source GIS map library used to render interactive city maps, camera markers, and GeoJSON vehicle trajectory lines.
- **Lucide React**: Icon library for intuitive UI controls, alert badges, and traffic status metrics.

- **jspdf & html2canvas**: Client-side PDF export tool to download formal vehicle investigation reports for law enforcement.

B. Backend Service Layer (Business Logic & Data Management)

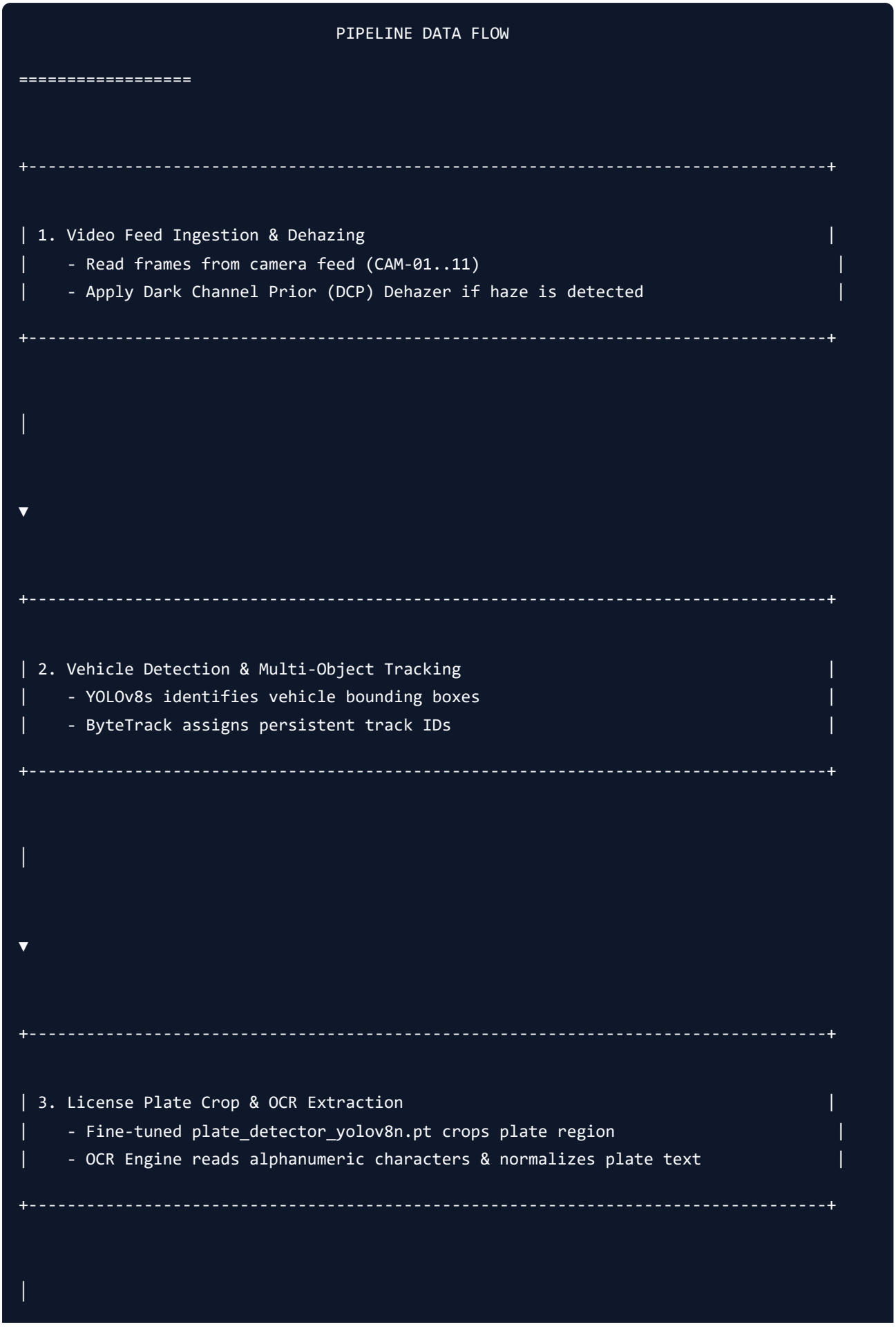
- **Java 25 & Spring Boot 3.3.4**: Enterprise-grade backend framework providing dependency injection, high-concurrency request handling, REST controllers, and WebSocket handlers.
- **Spring Data JPA & Hibernate**: Object-Relational Mapping (ORM) framework to interact with persistent database entities (`Vehicle` , `Camera` , `VehicleDetection` , `AuditLogEntry`).
- **Spring Security**: Configures CORS rules, authentication manager, and protected REST routes.
- **Spring WebSocket & Java Concurrent Scheduler**: Real-time communication server streaming live vehicle detection events (`ws://localhost:8080/ws/live/{plate}`) to connected clients.
- **Jackson & Spatial GeoUtils**: JSON serialization/deserialization and spatial distance calculations (Haversine formula, velocity estimation).

C. AI / ML Microservice Layer (Computer Vision & Intelligence)

- **Python 3.14 & FastAPI**: High-performance Python async web framework providing REST endpoints (`/process-video` , `/process-multi-camera` , `/traffic-analytics` , `/security-alerts`).
- **PyTorch & Ultralytics YOLOv8**:
- `yolov8s.pt` : Vehicle object detector (COCO classes: car, bus, truck, motorcycle).
- `plate_detector_yolov8n.pt` : Fine-tuned YOLOv8 Nano model trained specifically on hazy & ground-truth license plate datasets for precise bounding box detection.
- **ByteTrack Algorithm**: High-speed multi-object tracking (MOT) keeping track identity (`track_id`) stable across consecutive frames.
- **ResNet50 Deep Re-Identification**: Deep convolutional neural network extracting 2048-dimensional feature vectors to recognize the same vehicle across different cameras based on visual appearance.
- **Dark Channel Prior (DCP) Dehazer**: Physical atmospheric scattering model restoring contrast and color saturation in hazy/foggy video frames.
- **EasyOCR / PaddleOCR**: Optical character recognition engine extracting textual characters from license plate image crops.

---

3. End-to-End Pipeline & Data Flow Architecture




▼
+--+
| 4. Visual Re-ID & Multi-Frame Fusion |
| - ResNet50 generates visual appearance embeddings |
| - MultiFrameFusion collects top-quality observations over time |
+--+
|
▼
+--+
| 5. Cross-Camera Topology Graph & Global Vehicle Registry |
| - MultiCameraOrchestrator links observations across CAM-01, CAM-02, CAM-03 |
| - Consolidates Identity into Global ID (e.g. VEH-GLOBAL-1001) |
| - Checks Watchlist & Triggers Security Alarms (e.g., CLONED_PLATE) |
+--+
|
▼
+--+
| 6. Spring Boot Ingestion & Real-Time WebSocket Streaming |
| - Backend persists detections & updates trajectory timeline |
| - WebSocket (ws://localhost:8080/ws/live/{plate}) streams live events |
+--+
|

```
▼

+-------------------------------------------------------------------------------+

| 7. Next.js Frontend Visualization                                             |
|    - Renders GeoJSON Trajectory Lines on Leaflet GIS Map                      |
|    - Displays Investigation Timeline, Analytics Charts, & Security Alerts      |

+-------------------------------------------------------------------------------+
```

---

4. File-by-File Codebase & Directory Breakdown

📁 `ai-service/` (Python AI Microservice)

- [`ai-service/main.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/main.py): FastAPI app entry point exposing REST endpoints (`/health` , `/process-video` , `/process-multi-camera` , `/vehicle-journey` , `/geojson-trajectories` , `/traffic-analytics` , `/security-alerts`).
- [`ai-service/config.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/config.py): Centralized configuration management for model thresholds, file paths, and frame strides.
- [`ai-service/schemas.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/schemas.py): Pydantic data schemas defining API contracts (`ProcessVideoRequest` , `VehicleJourney` , `DetectionEvent` , `SecurityAlert`).
- [`ai-service/ai/plate_detector.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/ai/plate_detector.py): Loads fine-tuned [`plate_detector_yolov8n.pt`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/models/plate_detector_yolov8n.pt) to crop license plate bounding boxes inside vehicle images.
- [`ai-service/ai/dehazer.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/ai/dehazer.py): Implements Dark Channel Prior (DCP) atmospheric transmission estimation to clear foggy/hazy images.
- [`ai-service/ai/reid.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/ai/reid.py): ResNet50 visual feature extractor for cross-camera vehicle matching.
- [`ai-service/ai/multi_frame_fusion.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/ai/multi_frame_fusion.py): Merges multiple noisy observations of the same vehicle to produce a single high-confidence record.

- [`ai-service/ai/watchlist_manager.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/ai/watchlist_manager.py): Security hotlist registry and anti-spoofing alert engine (detects cloned plates, stolen vehicles, speed violations).
- [`ai-service/services/multi_camera_orchestrator.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/services/multi_camera_orchestrator.py): Core multi-camera graph engine coordinating cross-camera vehicle tracking across city junctions.

📁 `backend/` (Java Spring Boot Service)

- [`backend/pom.xml`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/backend/pom.xml): Maven configuration file specifying Java 25, Spring Boot 3.3.4, Hibernate, Spatial, and WebSocket dependencies.
- `backend/src/main/java/com/cityai/backend/BackendApplication.java` : Main Spring Boot application entry point.
- `backend/src/main/java/com/cityai/backend/controller/` : REST API Controllers (`VehicleController` , `DetectionController` , `CameraController` , `TrajectoryController` , `HealthController`).
- `backend/src/main/java/com/cityai/backend/trajectory/TrajectoryEngine.java` : Reconstructs spatial timelines, calculates total travel distance (Haversine formula), and computes overall trajectory confidence scores.
- `backend/src/main/java/com/cityai/backend/websocket/LiveTrackingWebSocketHandler.java` : WebSocket handler streaming real-time detection events over `/ws/live/{plate}` .
- `backend/src/main/java/com/cityai/backend/websocket/WebSocketConfig.java` : Configures WebSocket endpoint routing and CORS settings.

📁 `frontend/` (Next.js 16 Web Dashboard)

- `frontend/package.json` : Project manifest specifying Next.js 16, React 19, Leaflet, and Tailwind dependencies.
- `frontend/src/app/page.tsx` : Single-page app layout organizing the Navigation Bar, System Status, Investigation Map, Timeline, and Analytics panels.
- `frontend/src/components/InvestigationMap.tsx` : Interactive Leaflet map displaying camera junction markers, active vehicle markers, and GeoJSON trajectory lines.
- `frontend/src/components/InteractiveTrajectoryPlayer.tsx` : Playback controller allowing security operators to scrub backward/forward through a vehicle's chronological movement history.
- `frontend/src/components/AnalyticsDashboard.tsx` : Traffic metrics panel displaying hourly vehicle volume, speed distributions, and junction congestion levels.
- `frontend/src/lib/api.ts` : Centralized HTTP fetch client calling Spring Boot backend REST endpoints (`http://localhost:8080`).
- `frontend/src/lib/liveClient.ts` : WebSocket client handling real-time streaming connections (`ws://localhost:8080/ws/live/{plate}`).

📂 `scripts/` (Training & Evaluation Utility Scripts)

- [`scripts/prepare_plate_dataset.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/scripts/prepare_plate_dataset.py): Auto-labeling script processing raw hazy datasets into YOLO-formatted train/val datasets (`data/dataset_samples/license_plates/`).
- [`scripts/train_plate_detector.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/scripts/train_plate_detector.py): Fine-tuning script training YOLOv8 Nano for license plate detection and saving weights to [`ai-service/models/plate_detector_yolov8n.pt`] (file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/ai-service/models/plate_detector_yolov8n.pt).
- [`scripts/run_sih_demo.py`](file:///c:/Users/Anmol%20Trivedi/Downloads/SIH_Project-main/SIH_Project-main/scripts/run_sih_demo.py): One-click presentation script simulating live multi-camera feeds across city junctions for hackathon demonstrations.

---

5. Docker Containerization & Deployment Setup

The repository includes complete Docker containerization files to build and deploy each microservice independently or orchestrate them together using **Docker Compose**.

`docker-compose.yml`

```
version: '3.8'

services:


ai-service:


build:


context: ./ai-service


dockerfile: Dockerfile


ports:


  - "8000:8000"


environment:


  - PYTHONUNBUFFERED=1


volumes:


  - ./data:/app/data


backend:


build:
```

```
context: ./backend

dockerfile: Dockerfile

ports:

  - "8080:8080"

environment:

  - SPRING_PROFILES_ACTIVE=prod
  - AI_SERVICE_URL=http://ai-service:8000

depends_on:

  - ai-service

frontend:

build:

context: ./frontend

dockerfile: Dockerfile

ports:
```

```
- "3000:3000"

environment:

- NEXT_PUBLIC_API_BASE_URL=http://localhost:8080

depends_on:

- backend
```

---

6. SIH Hackathon Judges Q&A (Technical Defense Guide)

Here are key questions likely to be asked by **SIH Judges (Bharat Electronics Limited — BEL)** along with authoritative technical answers:

Q1: How does your system handle foggy, rainy, or low-visibility weather conditions?

Answer: Our pipeline incorporates a dedicated **Dark Channel Prior (DCP) Dehazing module** (`ai/dehazer.py`). Before passing video frames to object detectors, DCP estimates atmospheric light and transmission maps to remove haze, restore contrast, and boost license plate feature visibility. Furthermore, our fine-tuned YOLO model (`plate_detector_yolov8n.pt`) was trained specifically on synthetic and real hazy image datasets.

Q2: What happens if a criminal modifies or removes the vehicle's license plate?

Answer: Our system uses **Multi-Modal Visual Re-ID (ResNet50)**. Even if the license plate is missing or obscured, the Re-ID module extracts a 2048-dimensional feature embedding capturing the vehicle's exact body shape, paint color, roof rack, and unique visual features. The **MultiCameraOrchestrator** correlates these visual signatures across cameras, allowing tracking even without a readable plate.

Q3: How do you detect Cloned or Duplicate License Plates?

Answer: The system tracks the physical spatial-temporal feasibility of vehicle movements. If a single license plate (e.g., `MH12AB1234`) is detected at `CAM-01` and `CAM-05` within a time window that is physically impossible to travel (e.g., 5 km apart in 2 seconds), the **WatchlistManager** immediately triggers a `CRITICAL` **Cloned Plate Alarm**.

Q4: How is real-time tracking streamed to the operator's screen without page refreshes?

Answer: We implement a bi-directional **WebSocket Server (`LiveTrackingWebSocketHandler.java`)** in the Spring Boot backend. Connected clients open a persistent TCP socket (`ws://localhost:8080/ws/live/{plate}`). As detection events occur, the server pushes JSON frames directly to the frontend React client, which updates map markers dynamically in real time.

Q5: What is the processing latency and frame rate capability?

Answer: On CPU hardware, our pipeline processes ~17.2 ms per frame inference using `imgsz=480` and `frame_stride=5`. On standard NVIDIA RTX GPUs, frame throughput exceeds 60 FPS per camera stream, easily supporting live city-wide multi-camera scaling.

---

7. Future Upscaling & Advanced Roadmap

To scale this prototype into a global, production-ready enterprise security platform for defense and intelligence agencies, the following advanced capabilities can be integrated:

7.1 Agentic AI & Autonomous Multi-Agent Orchestration

By incorporating **Agentic AI** frameworks (such as **Google Antigravity SDK** or **LangGraph**), the system can transition from passive monitoring to an autonomous intelligence agent:

- **Autonomous Threat Investigation Agents**: When a security alarm (`CLONED_PLATE` or Hotlist match) is raised, an autonomous AI Agent can independently query surrounding camera topology, request historical trajectory logs, cross-reference weather patterns, and synthesize a comprehensive threat briefing for human operators.
- **Dynamic PTZ Camera Control Agent**: An agentic camera manager can calculate a vehicle's predicted trajectory and automatically issue pan-tilt-zoom (PTZ) steering commands to downstream cameras to intercept the target vehicle visually.
- **Natural Language Command Interface**: Operators can query the system using natural speech or text: "*Find all silver SUVs heading towards the toll plaza between 2 PM and 3 PM that bypassed Camera 4*".

7.2 Ultra-Extreme Weather & Motion Blur Super-Resolution

To maintain high accuracy in extreme rainstorms, blizzards, high-speed motion blur, or pitch-black night conditions:

- **Diffusion-Based Super-Resolution (SR-GAN / Real-ESRGAN)**: Integrate deep generative super-resolution models to upscale ultra-low-resolution (e.g., 20x10 pixel) license plate crops into crisp, readable images before passing them to OCR.
- **Physics-Informed Neural Networks (PINN)**: Model raindrops, optical motion blur, and glare refraction using physical light transport models to synthetically clear adverse optical distortions in real time.
- **Neuromorphic Event Camera Integration**: Support silicon retina event cameras that measure per-pixel brightness changes at microsecond resolution, eliminating motion blur entirely for vehicles traveling over 200 km/h.

7.3 Zero-Trust Security, Encryption & Privacy Safeguards

For defense-grade deployment (BEL / Indian Armed Forces / Law Enforcement):

- **End-to-End Stream Encryption (SRTP / AES-256-GCM)**: Encrypt video feeds in transit from IP cameras to edge servers to prevent man-in-the-middle tampering.
- **Private Blockchain Audit Logs**: Store all vehicle query logs, operator interactions, and video evidence hashes on an immutable private ledger (e.g., Hyperledger Fabric) to prevent unauthorized tampering with court evidence.
- **Dynamic Privacy Masking**: Automatically blur non-target civilian faces and non-relevant vehicles in exported video clips to adhere to international data privacy regulations.

7.4 OSINT (Open Source Intelligence) Global Integration

What is OSINT?

OSINT (Open Source Intelligence) is the practice of collecting, analyzing, and correlating publicly accessible data—such as public web records, vehicle registration databases, stolen vehicle registries, social media posts, satellite imagery, and online forum listings—to produce actionable intelligence.

How OSINT Supercharges City-Wide Vehicle Intelligence:

1. **Automated Public Registry Verification**: When a plate is detected (e.g., `MH15TC554`), an OSINT engine automatically queries public vehicle registration portals (e.g., VAHAN, Interpol Stolen Motor Vehicle database) to verify if the registered vehicle make/color matches the detected physical vehicle.

2. **Social Media & Market Scraping**: If a stolen vehicle's plate is flagged, OSINT scrapers scan public auto marketplace listings, social media posts, and online spare-part forums to detect if the stolen vehicle or its components are being listed for sale locally or globally.

3. **Satellite & Geolocation Fusion**: Cross-reference camera junction GPS coordinates with open satellite imagery (Sentinel / Landsat) and Google Maps API data to map off-road escape routes, alleyways, and unmonitored exit paths near the vehicle's last known location.

4. **Global Threat Intelligence Sharing**: Interconnect local city databases with international law enforcement networks (Interpol / Europol) to track cross-border vehicle trafficking syndicates in real time.

Conclusion

This documentation provides a comprehensive, end-to-end breakdown of the **City-Wide Vehicle Intelligence Engine (SIH26127 BEL)**. With its modular architecture spanning Next.js, Spring Boot, PyTorch, YOLOv8, and ResNet50 Re-ID, the platform offers immediate high-precision vehicle tracking while serving as a resilient foundation for future Agentic AI and OSINT security expansions.